

9강 Matplotlib



컴퓨팅사고와 데이터 분석 기초

한영신

공지사항

저희 수업을 비롯해 모든 수업의 강의영상은 해당 영상을 만들어서 업로드한 분에게 저작권이 있습니다. 강의영상을 무단으로 링크를 배포하거나 수업 외의 사용을 금합니다.

본 사이트에서 수업자료로 이용되는 저작물은 저작권법 제 25조 수업목적저작물 이용 보상금제도에 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. 약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개 공유 및 수업 목적외의 사용을 금지합니다. 해당 규정을 어기는 학생은 법적으로 처벌의 대상이 될 수 있음을 유의해 주시기 바랍니다.

본 강의교안의 저작권은 한영신입니다.

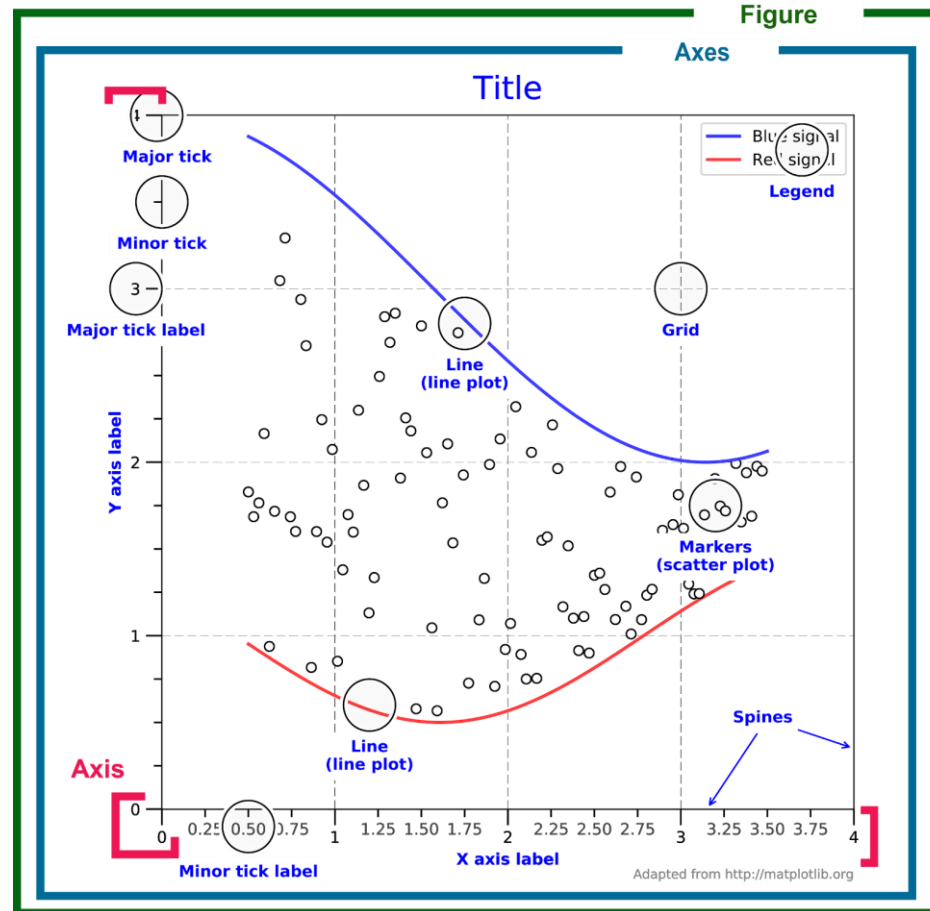
이 자료는 강의 보조자료로 제공되는 것으로 무단으로 전제하거나 배포하는 것을 금합니다 .



- Matplotlib
 - Python에서 데이터를 차트나 플롯으로 그려주는 라이브러리 패키지
 - 라인 플롯, 바 차트, 파이 차트, 히스토그램, Scatter Plot 등을 지원
- Matplotlib 사용법
 - 다음과 같이 Matplotlib.pyplot을 Import

```
from matplotlib import pyplot as plt
```

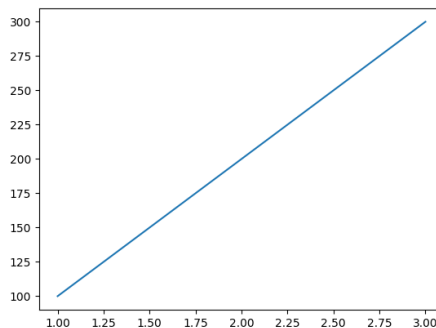
Matplotlib 용어



- Matplotlib.pyplot 사용 예제
 - X 축값 [1, 2, 3], Y 축값 [100, 200, 300]을 플롯에 도시
 - 실제 그림을 표시하는 함수인 `plt.show()`를 호출

```
import matplotlib.pyplot as plt
```

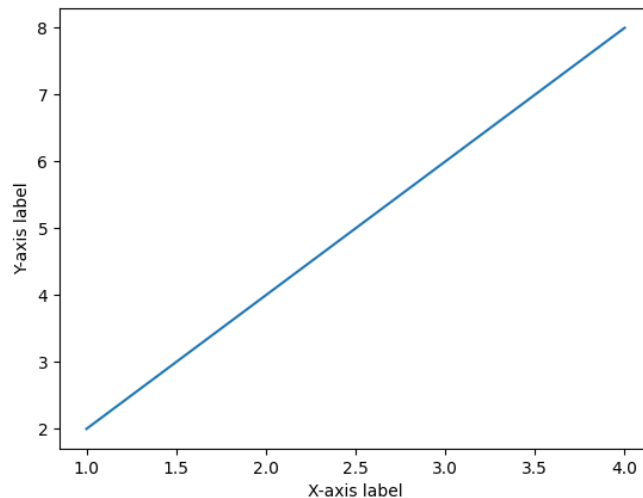
```
plt.plot([1, 2, 3, 4])  
plt.show()
```



레이블 (label)

- `xlabel()`, `ylabel()`을 이용하여 그래프의 x, y 축에 대한 레이블 표시가 가능

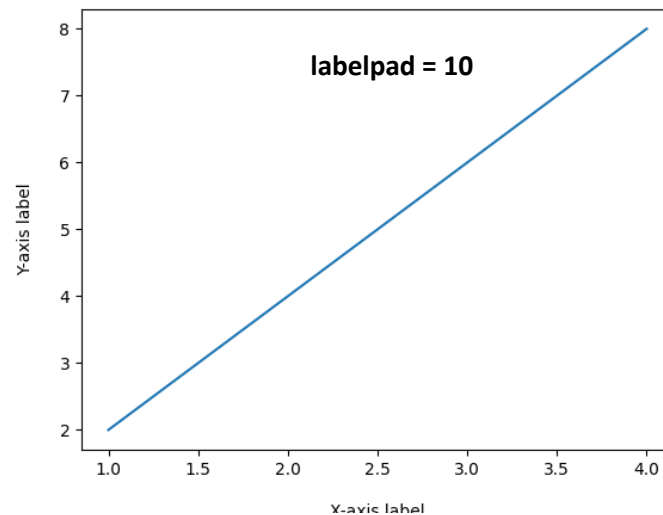
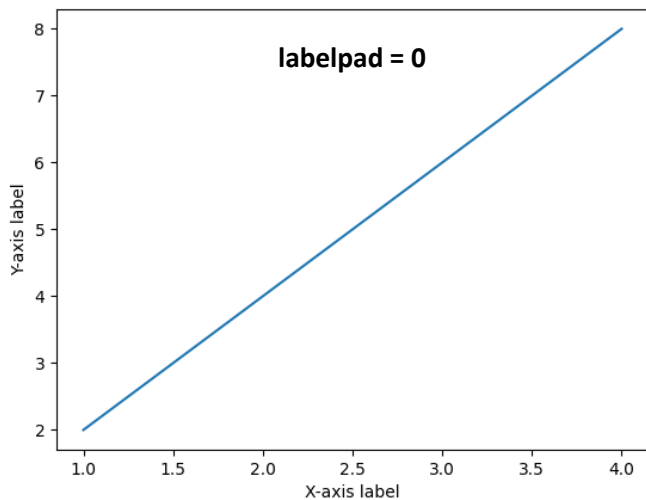
```
import matplotlib.pyplot as plt  
  
plt.plot([1, 2, 3, 4], [2, 4, 6, 8])  
plt.xlabel('X-axis label')  
plt.ylabel('Y-axis label')  
plt.show()
```



레이블 (label) - 여백 추가

- `labelpad`를 이용하여 레이블과 축 사이의 여백 추가 가능

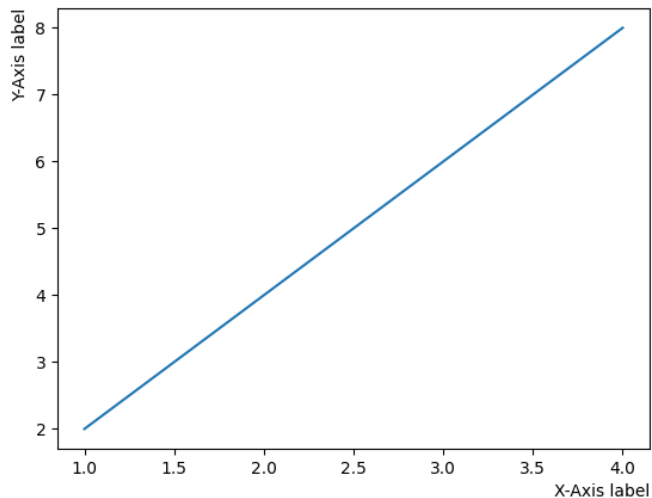
```
import matplotlib.pyplot as plt  
  
plt.plot([1, 2, 3, 4], [2, 4, 6, 8])  
plt.xlabel('X-axis label', labelpad=10)  
plt.ylabel('Y-axis label', labelpad=10)  
plt.show()
```



레이블 (label) - 위치 지정

- `loc=''`을 이용하여 원하는 위치에 레이블 지정 가능

```
import matplotlib.pyplot as plt  
  
plt.plot([1, 2, 3, 4], [2, 4, 6, 8])  
plt.xlabel('X-Axis label', loc='right')  
plt.ylabel('Y-Axis label', loc='top')  
plt.show()
```

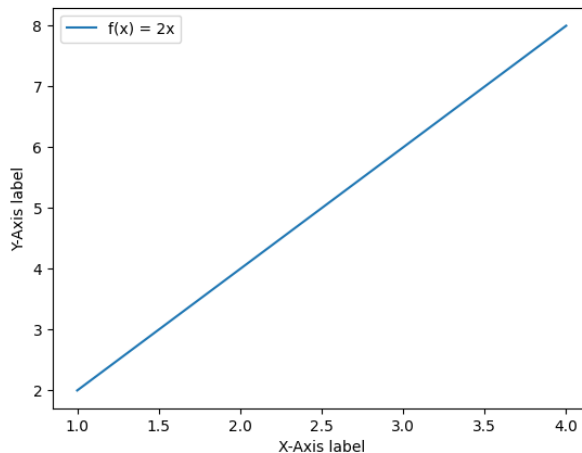


범례 (legend)

- 범례 : 그래프의 데이터 종류를 표시하기 위한 텍스트
- Matplotlib.pyplot 모듈의 legend() 함수를 이용

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정
# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')
# 범례 사용
plt.legend()
plt.show()
```

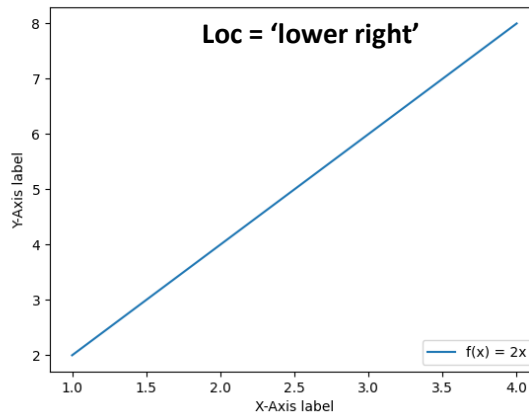
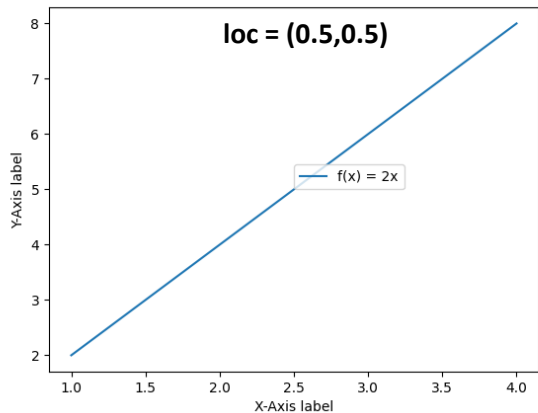


범례 (legend) - 위치 지정

- `legend()` 함수의 `loc` 파라미터를 이용해 범례가 표시될 위치 설정

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정
# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')
# 범례 사용
# loc(0,0) : plot 왼쪽 하단, loc(1,1) : plot 오른쪽 상단
plt.legend(loc=(0.5,0.5))
# plt.legend(loc = 'lower right')
plt.show()
```



Location String	Location Code
'best'	0
'upper right'	1
'upper left'	2
'lower left'	3
'lower right'	4
'right'	5
'center left'	6
'center right'	7
'lower center'	8
'upper center'	9
'center'	10

범례 (legend) - 열 개수 지정

- legend()함수의 ncol 파라미터를 이용하여 열 개수 지정 가능 (default : ncol=1)
- 여러 개의 범례를 plot할 때 사용 가능

```
import matplotlib.pyplot as plt
```

```
plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정  
plt.plot([1, 2, 3, 4], [4, 8, 12, 16], label='f(x) = 4x') # label을 통해 범례 지정
```

```
# 레이블 사용
```

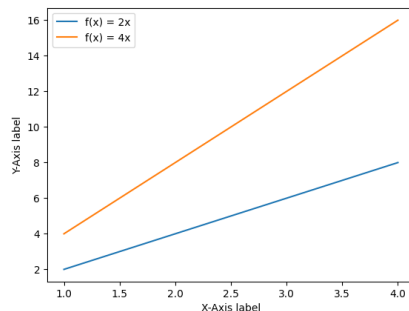
```
plt.xlabel('X-Axis label')
```

```
plt.ylabel('Y-Axis label')
```

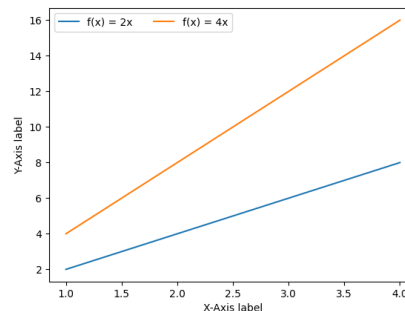
```
# 범례 사용
```

```
plt.legend(loc = 'upper left', ncol = 2)
```

```
plt.show()
```



ncol = 1



ncol = 2

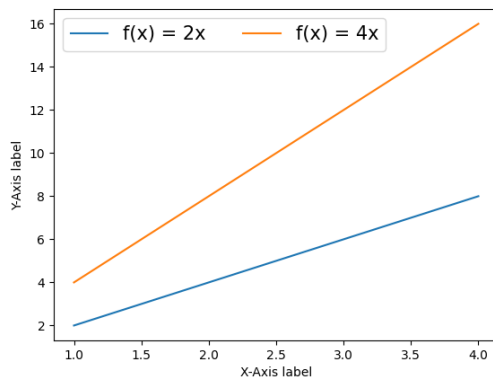
범례 (legend) - 폰트 사이즈

- `legend()` 함수의 `fontsize` 파라미터를 이용하여 설정 가능

```
import matplotlib.pyplot as plt

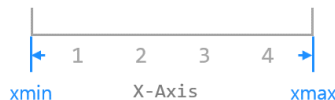
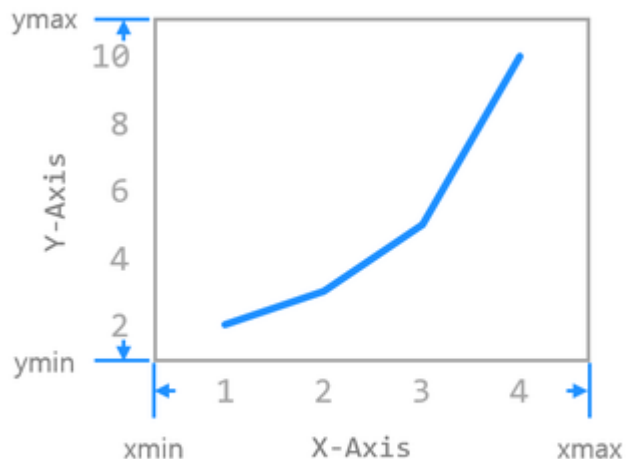
plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정
plt.plot([1, 2, 3, 4], [4, 8, 12, 16], label='f(x) = 4x') # label을 통해 범례 지정

# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')
# 범례 사용
plt.legend(loc = 'upper left', ncol = 2, fontsize = 15)
plt.show()
```



축 범위 지정

- Matplotlib.pyplot 모듈의 `xlim()`, `ylim()`, `axis()` 함수를 사용하여 x,y축 범위 지정



```
plt.xlim(xmin, xmax)  
plt.xlim((xmin, xmax))  
plt.xlim([xmin, xmax])
```



```
plt.ylim(ymin, ymax)  
plt.ylim((ymin, ymax))  
plt.ylim([ymin, ymax])
```

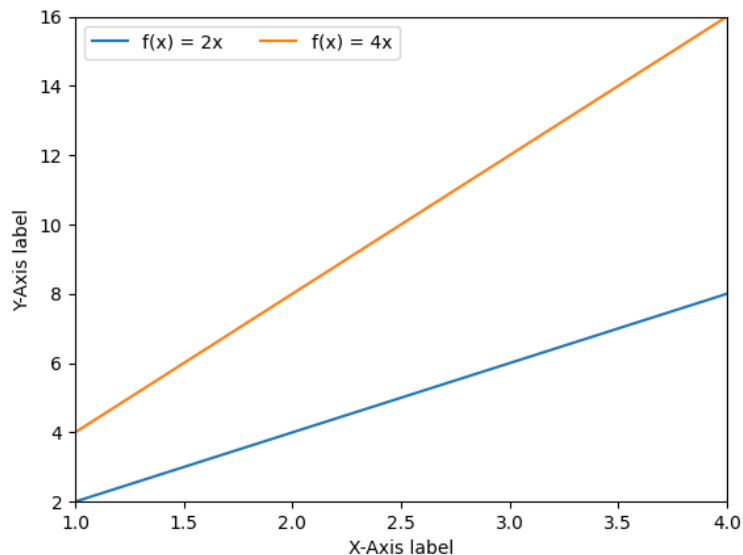
축 범위 지정 - xlim(), ylim()

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정
plt.plot([1, 2, 3, 4], [4, 8, 12, 16], label='f(x) = 4x') # label을 통해 범례 지정

# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')
# 범례 사용
plt.legend(loc = 'upper left', ncol = 2, fontsize = 10)
# 범위 지정
plt.xlim([1,4])
plt.ylim([2,16])

plt.show()
```



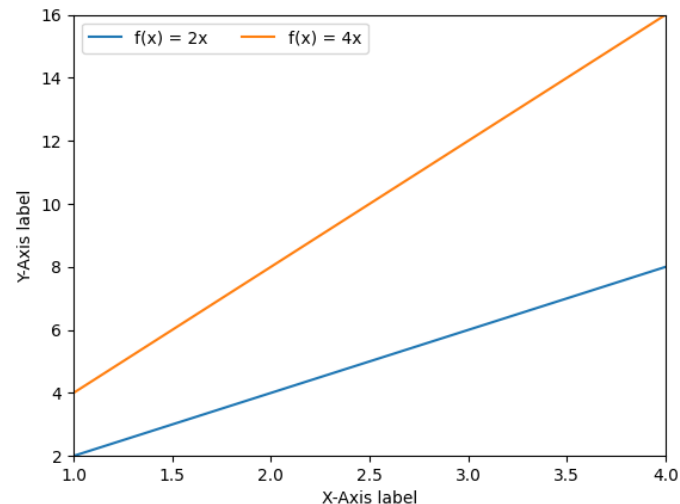
축 범위 지정 - axis()

- `axis([xmin, xmax, ymin, ymax])`

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정
plt.plot([1, 2, 3, 4], [4, 8, 12, 16], label='f(x) = 4x') # label을 통해 범례 지정

# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')
# 범례 사용
plt.legend(loc = 'upper left', ncol = 2, fontsize = 10)
# 범위 지정
plt.axis([1,4,2,16])
plt.show()
```



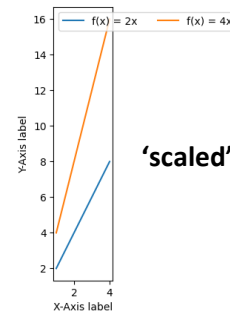
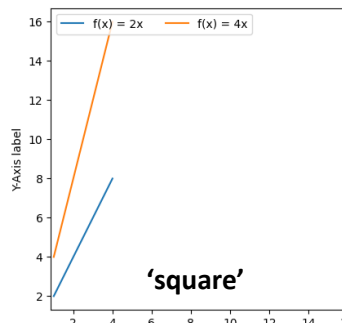
축 범위 지정 - axis() 옵션

- axis()은 다양한 옵션을 제공
- 'on' | 'off' | 'equal' | 'scaled' | 'tight' | 'auto' | 'normal' | 'image' | 'square'

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [2, 4, 6, 8], label='f(x) = 2x') # label을 통해 범례 지정
plt.plot([1, 2, 3, 4], [4, 8, 12, 16], label='f(x) = 4x') # label을 통해 범례 지정

# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')
# 범례 사용
plt.legend(loc = 'upper left', ncol = 2, fontsize = 10)
# 범위 지정
# plt.axis('square') # 축 길이 동일하게 표시
plt.axis('scaled') # X,Y축이 같은 길이 스케일로 표시
plt.show()
```



선 스타일 지정

- 데이터를 표현하기 위해 그려지는 선의 종류를 지정
- 아래의 방법 외에도 'linestyle' 파라미터를 이용한 방법도 있음

 Solid <code>plt.plot(x, y, '-')</code>	 Solid <code>plt.plot(x, y, linestyle='solid')</code>
 Dashed <code>plt.plot(x, y, '--')</code>	 Dashed <code>plt.plot(x, y, linestyle='dashed')</code>
 Dotted <code>plt.plot(x, y, '...')</code>	 Dotted <code>plt.plot(x, y, linestyle='dotted')</code>
 Dash-dot <code>plt.plot(x, y, '-.')</code>	 Dash-dot <code>plt.plot(x, y, linestyle='dashdot')</code>

```
import matplotlib.pyplot as plt
```

```
# 데이터 선정 및 선 스타일 지정
```

```
plt.plot([1, 2, 3, 4], [1, 1, 1, 1], '-', label='Solid')  
plt.plot([1, 2, 3, 4], [2, 2, 2, 2], '--', label='Dashed')  
plt.plot([1, 2, 3, 4], [3, 3, 3, 3], '...', label='Dotted')  
plt.plot([1, 2, 3, 4], [4, 4, 4, 4], '-.', label='Dash-dot')
```

```
# 레이블 사용
```

```
plt.xlabel('X-Axis label')
```

```
plt.ylabel('Y-Axis label')
```

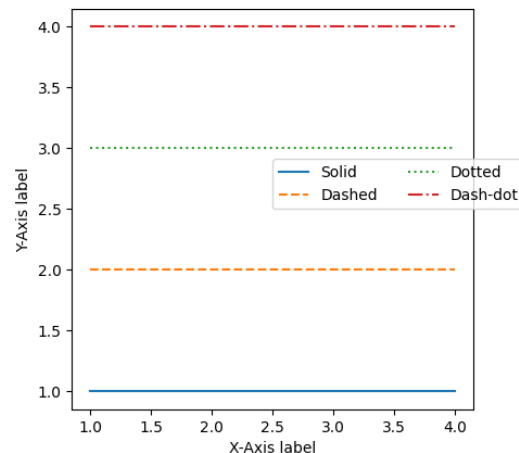
```
# 범례 사용
```

```
plt.legend(loc = (0.5, 0.5), ncol = 2, fontsize = 10)
```

```
# 범위 지정
```

```
plt.axis('square')
```

```
plt.show()
```



- 특별한 설정이 없을 시 실선으로 표시되지만 마커 형태 도시 가능

blue → circle
`plt.plot(x, y, 'bo')`

< 마커 설정 기본 형태 >

```
import matplotlib.pyplot as plt

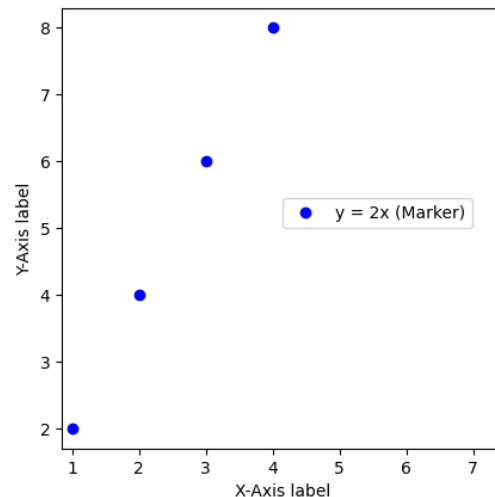
# 데이터 선정 및 선, 마커 스타일 지정
plt.plot([1, 2, 3, 4], [2, 4, 6, 8], 'bo', label='y = 2x (Marker)')

# 레이블 사용
plt.xlabel('X-Axis label')
plt.ylabel('Y-Axis label')

# 범례 사용
plt.legend(loc = (0.5, 0.5), ncol = 2, fontsize = 10)

# 범위 지정
plt.axis('square')

plt.show()
```



선, 마커, 색 스타일 지정 방식

blue circle solid line
`plt.plot(x, y, 'bo-')`

blue circle dashed line
`plt.plot(x, y, 'bo--')`

< 선, 마커, 색 설정 기본 형태 >

Colors

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

Line Styles

character	description
'_'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style

Markers

character	description
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
' '	vline marker
'_'	hline marker

선, 마커, 색 동시 스타일 지정

```
import matplotlib.pyplot as plt
```

```
# 데이터 선정 및 선,마커 스타일 지정
```

```
plt.plot([1, 2, 3, 4], [1, 2, 3, 4], 'bo-', label='y = x (Marker)')
```

```
plt.plot([1, 2, 3, 4], [2, 4, 6, 8], 'r^--', label='y = 2x (Marker)')
```

```
plt.plot([1, 2, 3, 4], [3, 6, 9, 12], 'k+:', label='y = 3x (Marker)')
```

```
plt.plot([1, 2, 3, 4], [4, 8, 12, 16], 'gx-.', label='y = 4x (Marker)')
```

```
# 레이블 사용
```

```
plt.xlabel('X-Axis label')
```

```
plt.ylabel('Y-Axis label')
```

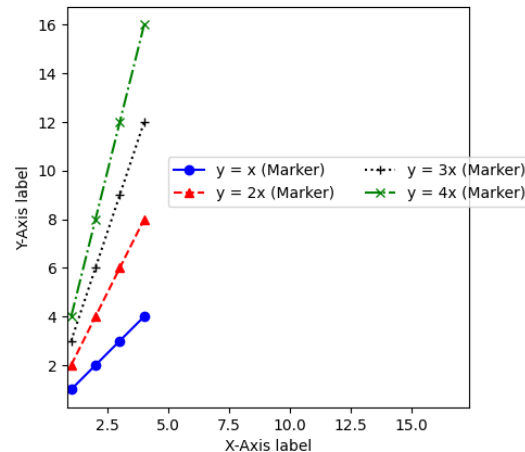
```
# 범례 사용
```

```
plt.legend(loc = (0.25, 0.5), ncol = 2, fontsize = 10)
```

```
# 범위 지정
```

```
plt.axis('square')
```

```
plt.show()
```

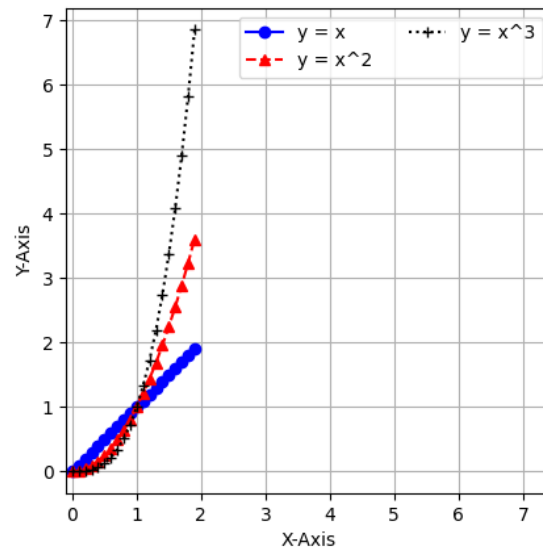


- Matplotlib.pyplot 모듈의 `grid()` 함수를 이용하여 다양한 그리드 설정 가능

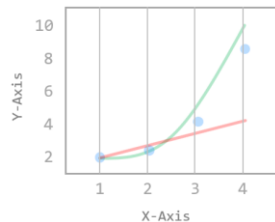
```
import matplotlib.pyplot as plt
import numpy as np

# 변수 설정
x = np.arange(0,2,0.1)
# 데이터 선정 및 선,마커 스타일 지정
plt.plot(x, x, 'bo-', label='y = x')
plt.plot(x, x**2, 'r^--', label='y = x^2')
plt.plot(x, x**3, 'k+.', label='y = x^3')
# 레이블 사용
plt.xlabel('X-Axis')
plt.ylabel('Y-Axis')
# 범례 사용
plt.legend(loc = 'best', ncol = 2, fontsize = 10)
# 범위 지정
plt.axis('square')
# 그리드 설정
plt.grid(True)

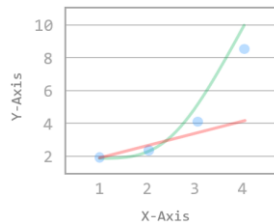
plt.show()
```



그리드 설정 - 축 지정



`plt.grid(True, axis='x')`



`plt.grid(True, axis='y')`

```
import matplotlib.pyplot as plt
import numpy as np
```

변수 설정

```
x = np.arange(0,2,0.1)
```

데이터 선정 및 선,마커 스타일 지정

```
plt.plot(x, x, 'bo-', label='y = x')
```

```
plt.plot(x, x**2, 'r^-', label='y = x^2')
```

```
plt.plot(x, x**3, 'k+.', label='y = x^3')
```

레이블 사용

```
plt.xlabel('X-Axis')
```

```
plt.ylabel('Y-Axis')
```

범례 사용

```
plt.legend(loc = 'best', ncol = 2, fontsize = 10)
```

범위 지정

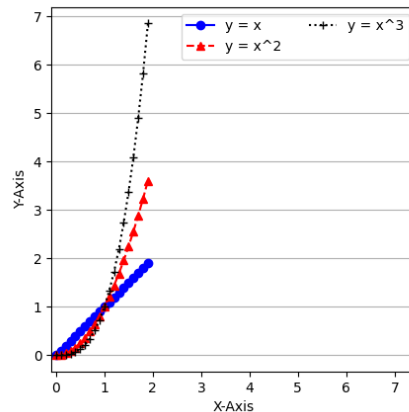
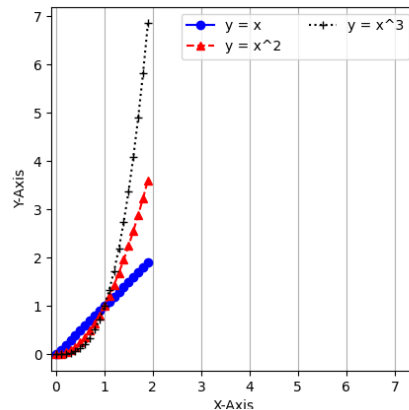
```
plt.axis('square')
```

그리드 설정

```
# plt.grid(True, axis='x') # 세로 방향 그리드
```

```
plt.grid(True, axis='y') # 가로 방향 그리드
```

```
plt.show()
```



- Matplotlib.pyplot 모듈의 `xticks()`, `yticks()` 함수를 이용하여 지정 가능
- `labels` 파라미터를 사용하여 눈금 레이블을 문자열 형태로 저장 가능

```
import matplotlib.pyplot as plt
import numpy as np
```

```
# 변수 설정
```

```
x = np.arange(0,2,0.2)
```

```
# 데이터 선정 및 선,마커 스타일 지정
```

```
plt.plot(x, x, 'bo-', label='y = x')
```

```
plt.plot(x, x**2, 'r^--', label='y = x^2')
```

```
plt.plot(x, x**3, 'k+:', label='y = x^3')
```

```
# 레이블 사용
```

```
plt.xlabel('X-Axis')
```

```
plt.ylabel('Y-Axis')
```

```
# 범례 사용
```

```
plt.legend(loc = 'best', ncol = 2, fontsize = 10)
```

```
# 범위 지정
```

```
# plt.axis('square')
```

```
# 그리드 설정
```

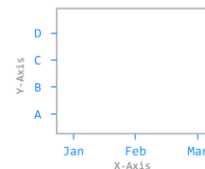
```
plt.grid(True)
```

```
# 눈금 설정
```

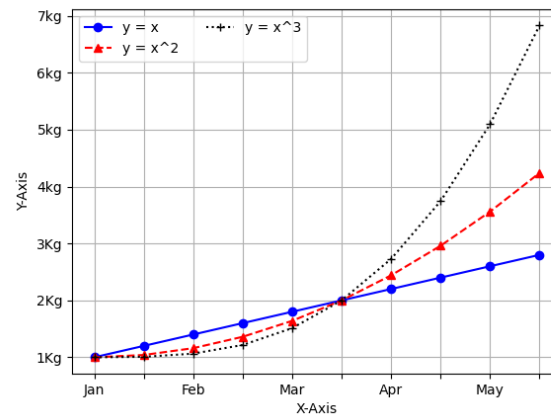
```
plt.xticks(np.arange(0, 2, 0.2), labels=['Jan', '', 'Feb', '', 'Mar', '', 'Apr', '', 'May', ''])
```

```
plt.yticks(np.arange(0, 7), labels=['1Kg', '2Kg', '3Kg', '4kg', '5kg', '6kg', '7kg'])
```

```
plt.show()
```



```
plt.xticks([0, 1, 2], label=['Jan', 'Feb', 'Mar'])
plt.yticks([1, 2, 3, 4], label=['A', 'B', 'C', 'D'])
```



- Matplotlib.pyplot 모듈의 title() 함수를 이용해 타이틀 지정
- loc = 'left', 'center', 'right'
- fontdict
 - 'fontsize'
 - 'fontweight' = {'normal', 'bold', 'heavy', 'light', 'ultrabold', 'ultralight'}

```
plt.title('Graph Title', loc='right', pad=20)
```

< 타이틀 제목, 위치, 여백 설정 >

```
plt.title('Graph Title', fontdict={...})
```

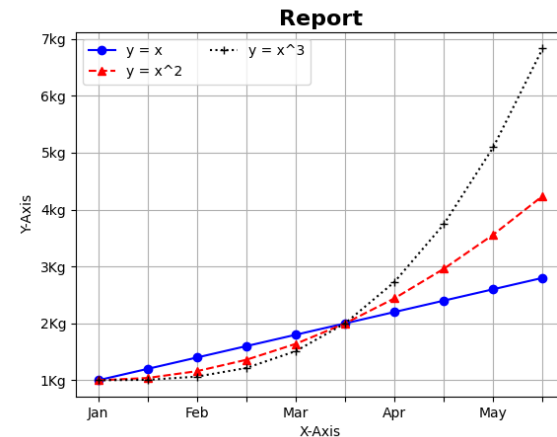
< 타이틀 폰트 지정 >

타이틀 설정



```
import matplotlib.pyplot as plt
import numpy as np

# 변수 설정
x = np.arange(0,2,0.2)
# 데이터 선정 및 선,마커 스타일 지정
plt.plot(x, x, 'bo-', label='y = x')
plt.plot(x, x**2, 'r^--', label='y = x^2')
plt.plot(x, x**3, 'k+.', label='y = x^3')
# 레이블 사용
plt.xlabel('X-Axis')
plt.ylabel('Y-Axis')
# 범례 사용
plt.legend(loc = 'best', ncol = 2, fontsize = 10)
# 범위 지정
# plt.axis('square')
# 그리드 설정
plt.grid(True)
# 눈금 설정
plt.xticks(np.arange(0, 2, 0.2), labels=['Jan', '', 'Feb', '', 'Mar', '', 'Apr', '', 'May', ''])
plt.yticks(np.arange(0, 7), labels=['1Kg', '2Kg', '3Kg', '4kg', '5kg', '6kg', '7kg'])
# 타이틀 설정
title_font = {'fontsize': 16, 'fontweight': 'bold'} # fontdict 정보 입력
plt.title('Report', fontdict=title_font, loc='center', pad=5)
plt.show()
```



여러 개의 그래프 그리기 - 수직 방향

- Matplotlib.pyplot 모듈의 subplot() 함수를 이용해 서브플롯 지정

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x1 = np.linspace(0.0, 5.0)
```

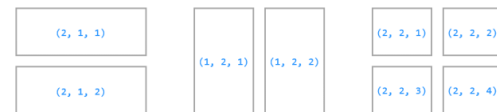
```
y1 = np.sin(2 * np.pi * x1)
```

```
y2 = np.exp(x1)
```

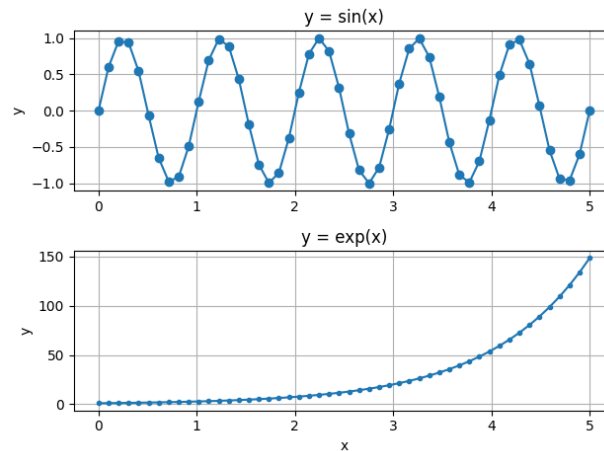
```
plt.subplot(2, 1, 1)          # nrow=2, ncol=1, index=1
plt.plot(x1, y1, 'o-')
plt.title('y = sin(x)')
plt.ylabel('y')
plt.grid(True)
```

```
plt.subplot(2, 1, 2)          # nrow=2, ncol=1, index=2
plt.plot(x1, y2, '-o')
plt.title('y = exp(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



plt.subplot(row, column, index)



여러 개의 그래프 그리기 - 수평 방향

- Matplotlib.pyplot 모듈의 subplot() 함수를 이용해 서브플롯 지정

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x1 = np.linspace(0.0, 5.0)
```

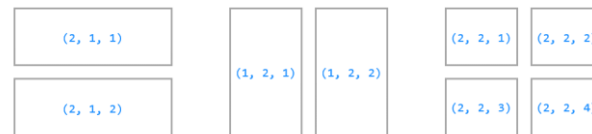
```
y1 = np.sin(2 * np.pi * x1)
```

```
y2 = np.exp(x1)
```

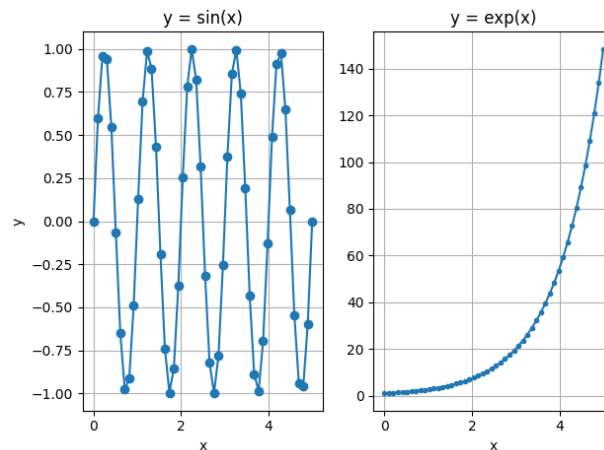
```
plt.subplot(1, 2, 1)           # nrows=1, ncols=2, index=1
plt.plot(x1, y1, 'o-')
plt.title('y = sin(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
```

```
plt.subplot(1, 2, 2)           # nrows=1, ncols=2, index=2
plt.plot(x1, y2, '-o')
plt.title('y = exp(x)')
plt.xlabel('x')
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



plt.subplot(row, column, index)



여러 개의 그래프 그리기 - 축 공유

- Subplot의 sharex 파라미터를 이용하여 축 공유 지정

```
import numpy as np
import matplotlib.pyplot as plt

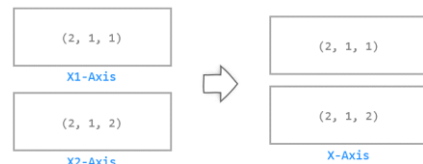
x1 = np.linspace(0.0, 5.0)
x2 = np.linspace(0.0, 10.0)

y1 = np.sin(2 * np.pi * x1)
y2 = np.cos(np.pi * x2) # 서로 주기가 다른 sin, cos 그래프

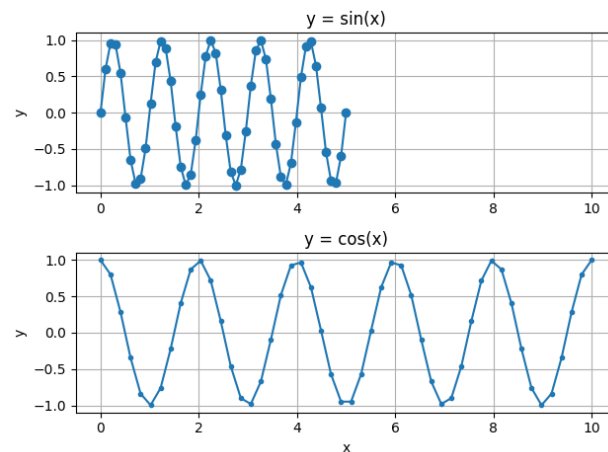
sp1 = plt.subplot(2, 1, 1) # nrows=2, ncols=1, index=1
plt.plot(x1, y1, 'o-')
plt.title('y = sin(x)')
plt.ylabel('y')
plt.grid(True)

sp2 = plt.subplot(2, 1, 2, sharex=sp1) # nrows=2, ncols=1, index=2
plt.plot(x2, y2, '-o')
plt.title('y = cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)

plt.tight_layout()
plt.show()
```



`plt.subplot(2, 1, 2, sharex=ax1)`

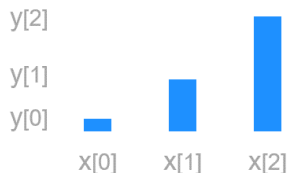


- Matplotlib.pyplot 모듈의 `bar()` 함수를 이용하여 막대 그래프 지정
 - `width` : 막대의 폭 지정 (default = 0.8)
 - `align` : 눈금, 막대의 위치 조절 ('edge' → 눈금이 막대의 왼쪽 끝에 표시)
 - `edgecolor` : 막대 테두리의 색 지정
 - `linewidth` : 테두리의 두께 지정



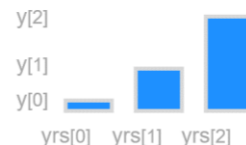
```
plt.bar(x, y, color=['r', 'g', 'b'])
```

< 막대그래프 색 지정 >



```
plt.bar(x, y, width=0.4)
```

< 막대그래프 폭 지정 >



```
plt.bar(x, y, align='edge', edgecolor='lightgray',  
        linewidth='5', tick_label=yrs)
```

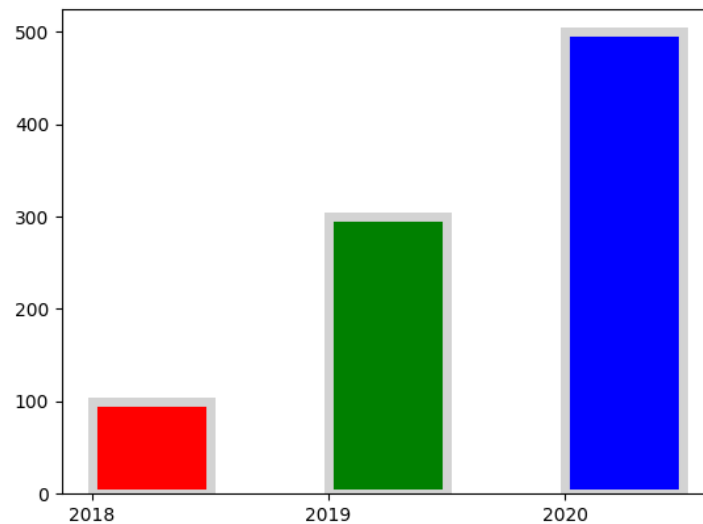
< 막대그래프 스타일 꾸미기 >

막대 그래프

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(3)
years = ['2018', '2019', '2020']
values = [100, 300, 500]

plt.bar(x, values, color=['r','g','b'],width=0.5, align='edge', edgecolor='lightgray',
        linewidth=5, tick_label=years)
plt.show()
```



산점도(Scatter plot)

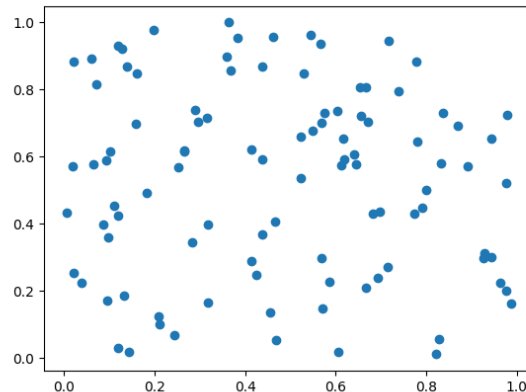
- 산점도
 - 두 변수의 상관 관계를 직교 좌표계의 평면에 점으로 표현하는 그래프
 - Matplotlib.pyplot 모듈의 scatter() 함수를 이용하여 도시

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)

n = 100
x = np.random.rand(n)
y = np.random.rand(n)

plt.scatter(x, y)
plt.show()
```



산점도(Scatter plot) - 색상, 크기

- 산점도
 - 두 변수의 상관 관계를 직교 좌표계의 평면에 점으로 표현하는 그래프
 - Matplotlib.pyplot 모듈의 scatter() 함수를 이용하여 도시

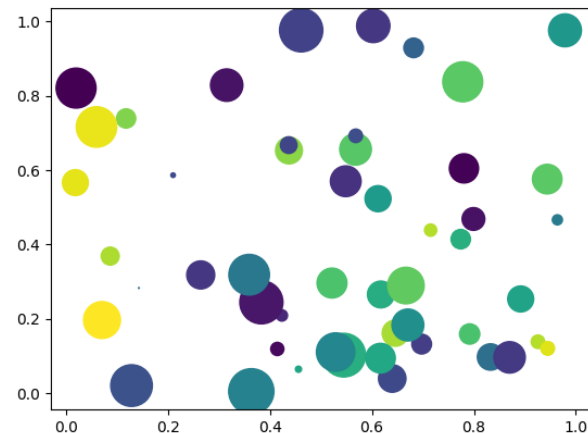
size color
↓ ↓
`plt.scatter(x, y, s=area, c=colors)`

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)

n = 50
x = np.random.rand(n)
y = np.random.rand(n)
area = (30 * np.random.rand(n))**2
colors = np.random.rand(n)

plt.scatter(x, y, s=area, c=colors)
plt.show()
```



산점도(Scatter plot) - 투명도, 컬러맵

- 투명도, 컬러맵
 - Alpha : 0~1 사이의 값을 입력, 마커의 투명도 설정
 - Cmap : 컬러맵에 해당하는 문자열 지정

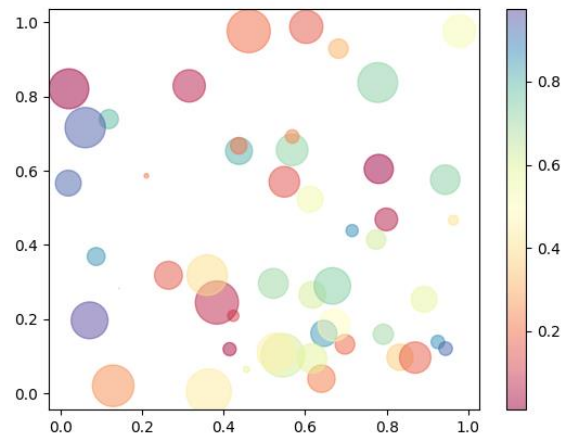
transparency colormap
↓ ↓
`plt.scatter(x, y, alpha=0.5, cmap='Spectral')`

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)

n = 50
x = np.random.rand(n)
y = np.random.rand(n)
area = (30 * np.random.rand(n))**2
colors = np.random.rand(n)

plt.scatter(x, y, s=area, c=colors, alpha=0.5, cmap='Spectral')
plt.colorbar()
plt.show()
```



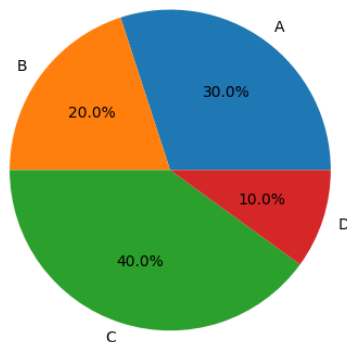
- matplotlib.pyplot의 pie() 함수를 사용하여 지정 가능
 - autopct: 부채꼴 안에 표시될 숫자의 형식 지정

```
import matplotlib.pyplot as plt
```

```
ratio = [30, 20, 40, 10]
```

```
labels = ['A', 'B', 'C', 'D']
```

```
plt.pie(ratio, labels=labels, autopct='%1f%%') # 소수점 한자리 까지 표시  
plt.show()
```



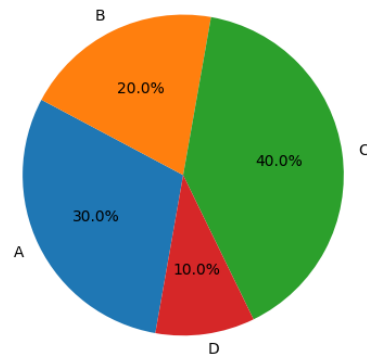
파이 차트 - 시작 각도, 방향 설정

- Startangle : 부채꼴이 그려지는 시작 각도 설정
 - default = 0 deg, 양의 방향 (x축 기준)
- Counterclock : true, false로 설정 (false = 시계 방향)

```
import matplotlib.pyplot as plt

ratio = [30, 20, 40, 10]
labels = ['A', 'B', 'C', 'D']

plt.pie(ratio, labels=labels, autopct='%1f%%', startangle=260, counterclock=False)
plt.show()
```



파이 차트 - 중심에서 벗어나는 정도 설정

- Explode : 부채꼴이 파이 차트의 중심에서 벗어나는 정도를 설정
 - 0.1 : 반지름의 10%

```
import matplotlib.pyplot as plt

ratio = [34, 32, 16, 18]
labels = ['A', 'B', 'C', 'D']
explode = [0, 0.10, 0, 0.10]

plt.pie(ratio, labels=labels, autopct='%1f%%', startangle=260,
        counterclock=False, explode=explode)
plt.show()
```

